OhioT1DM Patient DATA

Analysis Report

May 2026

Prof. Armando Ruggeri



Università
degli Studi di
Messina

Table of Contents

1. Introduction
2. Data Source Description
3. Phase 1: XML Parsing and CSV Conversion
4. Phase 2: Timestamp Standardization (ISO 8601)
5. Phase 3: Database Schema Design
6. Phase 4: Cross-Reference Queries with Timestamp Windowing
    - Query 1: Glucose at Mealtime
    - Query 2: Post-Meal Glucose Response
    - Query 3: Bolus Insulin Around Meals
    - Query 4: Finger Stick vs CGM
    - Query 5: Glucose During Sleep
    - Query 6: Glucose at Bolus Time
    - Query 7: Exercise Impact on Glucose
    - Query 8: Wearable Sensors at Meals
7. Conclusion

1. Introduction

The project transforms raw XML patient files into a structured relational database and implements cross-reference queries that connect multiple clinical data streams using timestamp-based windowing.

Type 1 Diabetes Mellitus (T1DM) is a chronic autoimmune condition in which the pancreas produces little or no insulin. Patients with T1DM must continuously monitor their blood glucose levels and administer insulin to maintain glucose within a safe range (typically 70-180 mg/dL). Modern diabetes management generates multiple streams of time-series data: continuous glucose monitors (CGM) record readings every five minutes, insulin pumps log every basal rate change and bolus injection, and wearable devices capture physiological signals such as skin temperature, galvanic skin response, and accelerometer data.

There is a challenge where each data stream is recorded independently with its own timestamps, sampling rates, and formats. A glucose reading at 19:06:45 and a meal logged at 19:00:00 are clearly related, but without cross-referencing, they appear as isolated events in separate tables. This project builds the infrastructure to bridge that gap.

The project consists of five phases: (1) parsing raw XML files and extracting each data section into clean CSV files, (2) converting non-standard timestamps to ISO 8601 format, (3) designing a normalized relational database schema, (4) populating the database and verifying data integrity, and (5) implementing cross-reference queries that use timestamp windowing to join records across data streams. The windowing approach uses tolerances of plus or minus 1 to 5 minutes to account for the fact that manually recorded measurements rarely share exact timestamps.

2. Data Source Description

The OhioT1DM dataset contains clinical and physiological data for six patients with Type 1 Diabetes, identified by numeric IDs: 540, 544, 552, 567, 584, and 596. Each patient's data is stored in a single XML file following a consistent hierarchical structure. The root element is a <patient> tag containing attributes for patient ID, weight (in kg), and insulin type (Humalog or Novalog).

2.1 XML File Structure

Each XML file contains up to 19 data sections nested under the <patient> root element. Some sections contain thousands of events (glucose readings, accelerometer data), while others may be empty depending on the patient's reporting. The sections fall into three functional categories:

Insulin Management

The glucose_level section contains CGM readings recorded at approximately five-minute intervals, with each event carrying a timestamp and a glucose value in mg/dL. The finger_stick section holds manual blood glucose checks performed by the patient using a traditional glucometer. The basal section records the scheduled background insulin delivery rate, while temp_basal captures temporary overrides to the basal rate, stored with both a start timestamp (ts_begin) and an end timestamp (ts_end). The bolus section logs discrete insulin doses administered for meals or corrections, with attributes for start time, end time, bolus type (normal, normal dual, or square dual), and dose in units.

Patient-Reported Events

The meal section records eating events with a timestamp, meal type (Breakfast, Lunch, Dinner, Snack, or HypoCorrection), and carbohydrate count in grams. The sleep section logs self-reported sleep periods with start and end times and a quality rating. The work section captures work shifts with start/end times and an intensity rating. The exercise section records physical activity with timestamp, intensity, type, duration in minutes, and whether the activity was competitive. Additional sections include stressors, hypo_event (hypoglycemic episodes with associated symptoms), and illness.

Wearable Sensor Data (Basis Band)

The Basis wearable band provides high-frequency physiological data recorded at one-minute intervals. The basis_gsr section captures galvanic skin response (a measure of electrodermal activity related to stress and arousal). The basis_skin_temperature section records skin surface temperature in degrees Fahrenheit. The basis_sleep section provides algorithmically detected sleep periods with start time, end time, quality score, and sleep type. The acceleration section captures accelerometer magnitude data reflecting physical movement.

3. XML Parsing and CSV Conversion

The implementation was developed in a Jupyter Notebook (Python) using the xml.etree.ElementTree module for parsing and pandas for DataFrame construction and CSV export.

3.1 Parsing Strategy

Each XML file was parsed by iterating over the child elements of the <patient> root. For each recognized section tag (glucose_level, finger_stick, basal, etc.), a dedicated parser function extracted the relevant attributes from every <event> element within that section. Different sections required different parsers because their XML structures vary:

Simple timestamp-value sections (glucose_level, finger_stick, basal, basis_gsr, basis_skin_temperature, acceleration) use events with two attributes: ts (timestamp) and value. These were extracted into two-column DataFrames with columns timestamp and value.

Range-based sections (temp_basal, sleep, work) use events with ts_begin and ts_end attributes defining a time interval. The bolus section extends this pattern with additional type and dose attributes. The meal section uses a single timestamp with type and carbs attributes. The exercise section includes timestamp, intensity, type, duration, and competitive attributes. The basis_sleep section uses tbegin and tend (note the different naming convention) along with quality and type.

3.2 Patient ID Injection

A critical design decision was to inject the patient_id as the first column in every CSV file. Although each patient's data resides in its own folder, including the patient identifier directly in the data ensures that CSV files are self-describing and can be loaded into a shared database table without ambiguity. The patient ID is extracted from the id attribute of the XML root <patient> element and inserted as the first column of every DataFrame before export.

3.3 Output Structure

The output is organized as a ZIP archive containing one folder per patient. Each folder is named using the convention "Patient {ID}" (e.g., Patient 540, Patient 544) and contains individual CSV files for each non-empty data section. A patient_info.csv file in each folder captures the patient-level metadata (patient_id, weight, insulin_type). Empty XML sections (self-closing tags with no child events) are skipped, so patients only receive CSV files for data streams that contain records.

4. Timestamp Standardization (ISO 8601)

The raw XML files use a non-standard timestamp format: DD-MM-YYYY HH:MM:SS (e.g., 04-07-2027 00:01:44). This European day-first format is ambiguous in international contexts and incompatible with most database systems, which expect timestamps to be sortable as strings. A timestamp like 04-07-2027 could be interpreted as April 7th or July 4th depending on locale conventions.

4.1 Conversion to ISO 8601

All timestamps were converted to ISO 8601 format: YYYY-MM-DD HH:MM:SS (e.g., 2027-07-04 00:01:44). This format is unambiguous, lexicographically sortable, and natively supported by PostgreSQL, and virtually every database engine.

The conversion function uses Python's datetime module:

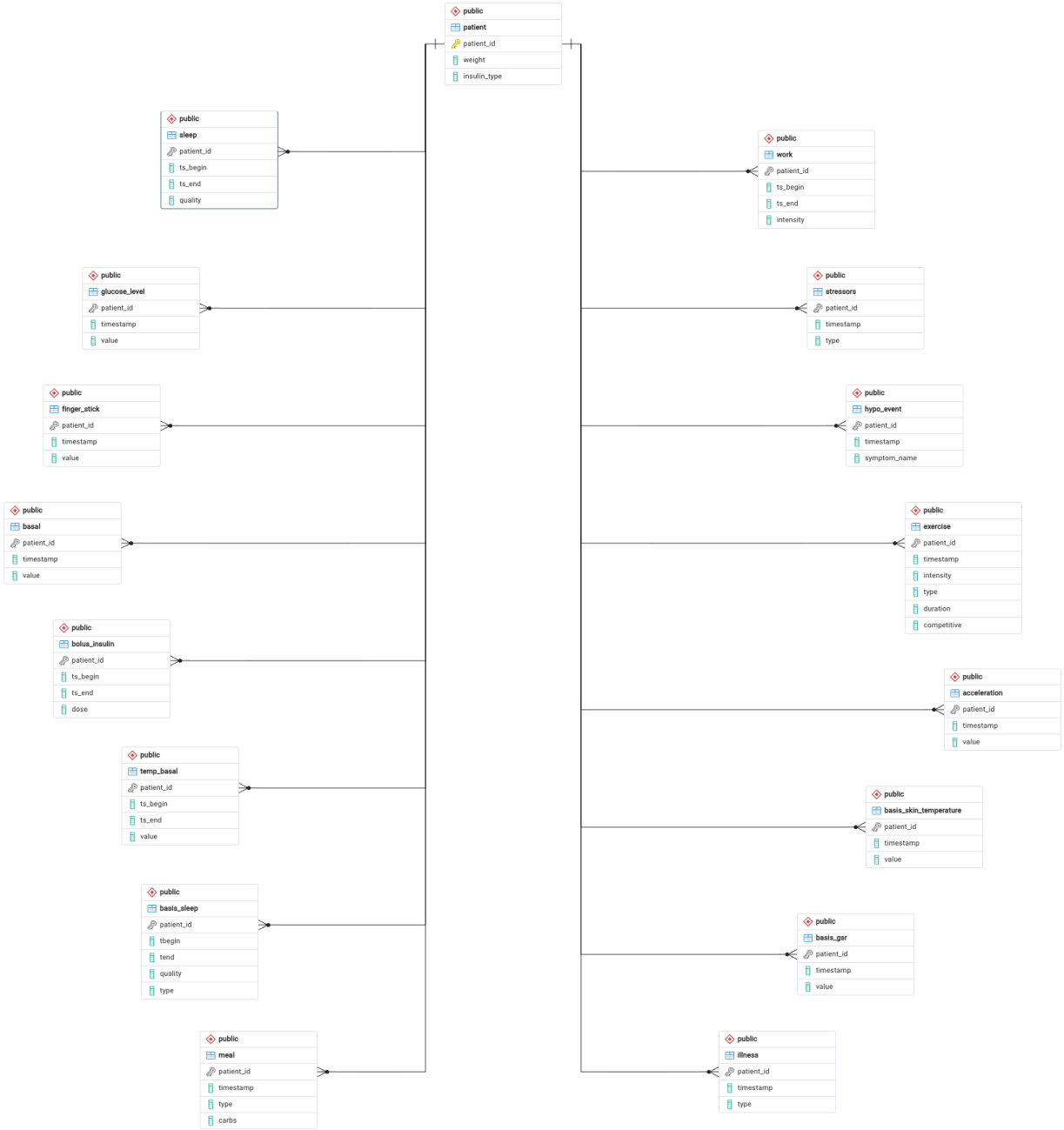
```
from datetime import datetime


def convert_timestamp(ts_str: str) -> str:
    try:
        dt = datetime.strptime(ts_str.strip(), '%d-%m-%Y %H:%M:%S')
        return dt.strftime('%Y-%m-%d %H:%M:%S')
    except ValueError:
        return ts_str
```

The try-except block ensures robustness: if a timestamp does not match the expected format, it is passed through unchanged rather than causing to fail.

5. Database Schema Design

The database schema was designed as a normalized relational model with the patient table as the central entity. All 16 data tables reference patient(patient_id) through foreign key constraints, enforcing referential integrity and ensuring that no orphaned records can exist in the system.



5.1 Schema Definition

The patient table serves as the primary reference table:

```
CREATE TABLE patient (
    patient_id    INTEGER PRIMARY KEY,
    weight        REAL,
    insulin_type  TEXT
);
```

Each data table follows the same pattern, referencing the patient table via foreign key. For example, the glucose_level and bolus_insulin tables:

```
CREATE TABLE glucose_level (
    patient_id  INTEGER NOT NULL,
    timestamp   TEXT    NOT NULL,
    value       REAL,
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
);
CREATE INDEX idx_glucose_patient_ts
    ON glucose_level(patient_id, timestamp);
CREATE TABLE bolus_insulin (
    patient_id  INTEGER NOT NULL,
    ts_begin    TEXT    NOT NULL,
    ts_end      TEXT    NOT NULL,
    type        TEXT,
    dose        REAL,
    FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
);
CREATE INDEX idx_bolus_patient_ts
    ON bolus_insulin(patient_id, ts_begin);
```

5.3 Complete Table Inventory

Table Name	Columns	Index
patient	patient_id (PK), weight, insulin_type	Primary key
glucose_level	patient_id (FK), timestamp, value	(patient_id, timestamp)
finger_stick	patient_id (FK), timestamp, value	(patient_id, timestamp)
basal	patient_id (FK), timestamp, value	(patient_id, timestamp)
temp_basal	patient_id (FK), ts_begin, ts_end, value	(patient_id, ts_begin)
bolus_insulin	patient_id (FK), ts_begin, ts_end, type, dose	(patient_id, ts_begin)
meal	patient_id (FK), timestamp, type, carbs	(patient_id, timestamp)
sleep	patient_id (FK), ts_begin, ts_end, quality	(patient_id, ts_begin)
work	patient_id (FK), ts_begin, ts_end, intensity	(patient_id, ts_begin)
hypo_event	patient_id (FK), timestamp, symptom_name	(patient_id, timestamp)
exercise	patient_id (FK), timestamp, intensity, type, duration, competitive	(patient_id, timestamp)
basis_gsr	patient_id (FK), timestamp, value	(patient_id, timestamp)
basis_skin_temperature	patient_id (FK), timestamp, value	(patient_id, timestamp)
basis_sleep	patient_id (FK), tbegin, tend, quality, type	(patient_id, tbegin)
acceleration	patient_id (FK), timestamp, value	(patient_id, timestamp)

6. Cross-Reference Queries with Timestamp Windowing

6.1 The Problem of Timestamp Alignment

The central analytical challenge in this dataset is that different data streams are recorded at different times and frequencies. The CGM records glucose every five minutes (e.g., 19:01:45, 19:06:45, 19:11:45). Meals are logged at round times (e.g., 19:00:00). Bolus injections are recorded to the second (e.g., 19:15:08). Wearable sensors sample every minute. Finding exact timestamp matches across these tables is virtually impossible.

The solution is timestamp windowing: instead of requiring exact matches, we define a tolerance window (e.g., plus or minus 5 minutes) and join any records whose timestamps fall within that window.

6.2 Window Size Selection

Different cross-references require different window sizes based on the clinical context and the sampling frequencies of the data streams involved:

Window Size	Use Case	Description
+/- 1 minute	Wearable sensors at meals	Basis band records every minute, so a tight window finds an exact or near-exact match
+/- 5 minutes	Glucose at meal/bolus/exercise time	CGM records every 5 minutes, so this window guarantees at least one reading
+/- 30 minutes	Bolus insulin around meals	Patients inject within 30 minutes of eating (before, during, or after)
Full range	Glucose during sleep/work	Glucose must fall inside the entire period, not near a single point

6.3 SQL Windowing Mechanism

The windowing is implemented using the SQL BETWEEN operator on casted timestamps. The general pattern for a plus or minus 5-minute window is:

```
JOIN glucose_level g
  ON g.patient_id = m.patient_id
 AND g.timestamp::timestamp
     BETWEEN m.timestamp::timestamp - INTERVAL '5 minutes'
         AND m.timestamp::timestamp + INTERVAL '5 minutes'
```

The ::timestamp cast converts the TEXT-stored ISO 8601 string to a PostgreSQL TIMESTAMP type, enabling arithmetic with INTERVAL values. The BETWEEN clause then defines the window boundaries. Results are typically ordered by the absolute offset from the target time, so the closest match appears first.

6.4 Query 1: Glucose at Mealtime

File: 01_glucose_at_meal_time.csv

This query finds the closest CGM reading to each meal event within a plus or minus 5-minute window. It answers the fundamental clinical question: what was the patient's blood sugar when they sat down to eat? The results are sorted by the absolute offset between the meal and glucose timestamps, so the first row per meal is always the closest match. The offset_minutes column shows the exact time difference, with negative values indicating the glucose reading preceded the meal and positive values indicating it followed.

```
SELECT
m.patient_id, m.timestamp AS meal_time,
m.type AS meal_type, m.carbs,
g.timestamp AS glucose_time,
g.value AS glucose_at_meal,


EXTRACT(EPOCH FROM
(g.timestamp::timestamp - m.timestamp::timestamp)) / 60.0 AS offset_minutes

FROM meal m
JOIN glucose_level g ON g.patient_id = m.patient_id AND g.timestamp::timestamp
BETWEEN m.timestamp::timestamp - INTERVAL '5 minutes'
AND m.timestamp::timestamp + INTERVAL '5 minutes'

ORDER BY
m.patient_id,
m.timestamp,
ABS(EXTRACT(EPOCH FROM (g.timestamp::timestamp - m.timestamp::timestamp)));
```

	patient_id integer	meal_time timestamp without time zone	meal_type text	carbs real	glucose_time timestamp without time zone	glucose_at_meal real	offset_minutes numeric
1	540	2027-07-04 09:00:00	Breakfast	40	2027-07-04 09:01:45	90	1.7500000000000000
2	540	2027-07-04 09:00:00	Breakfast	40	2027-07-04 08:56:45	89	-3.2500000000000000
3	540	2027-07-04 12:20:00	Lunch	30	2027-07-04 12:21:45	83	1.7500000000000000
4	540	2027-07-04 12:20:00	Lunch	30	2027-07-04 12:16:45	86	-3.2500000000000000
5	540	2027-07-04 19:00:00	Dinner	100	2027-07-04 19:01:46	144	1.7666666666666667
6	540	2027-07-04 19:00:00	Dinner	100	2027-07-04 18:56:46	148	-3.2333333333333333
7	540	2027-07-05 09:00:00	Breakfast	20	2027-07-05 09:01:49	58	1.8166666666666667
8	540	2027-07-05 09:00:00	Breakfast	20	2027-07-05 08:56:49	61	-3.1833333333333333
9	540	2027-07-05 11:30:00	Lunch	90	2027-07-05 11:31:50	76	1.8333333333333333
10	540	2027-07-05 11:30:00	Lunch	90	2027-07-05 11:26:50	76	-3.1666666666666667
11	540	2027-07-05 18:15:00	Dinner	67	2027-07-05 18:16:50	67	1.8333333333333333
12	540	2027-07-05 18:15:00	Dinner	67	2027-07-05 18:11:50	69	-3.1666666666666667
13	540	2027-07-06 08:00:00	Breakfast	90	2027-07-06 08:01:53	104	1.8833333333333333
14	540	2027-07-06 08:00:00	Breakfast	90	2027-07-06 07:56:53	106	-3.1166666666666667
15	540	2027-07-06 13:30:00	Lunch	45	2027-07-06 13:31:54	94	1.9000000000000000
16	540	2027-07-06 13:30:00	Lunch	45	2027-07-06 13:26:54	95	-3.1000000000000000

6.5 Query 2: Post-Meal Glucose Response

File: 02_post_meal_glucose_response.csv

This is the core clinical query. For each meal, it tracks the glucose trajectory at 0, +30, +60, +90, and +120 minutes after eating. This reveals the after-meal glucose response: does glucose spike sharply at +30 minutes? Does it peak at +60 or +90? How quickly does it return to baseline by +120 minutes? Without this cross-reference, we just see "ate 100g carbs" in one place and "glucose was 245" in another with no proven connection between them. This query provides that proof by placing both events on the same timeline.

The CROSS JOIN with a VALUES list generates the five-time offsets. For each offset, the join finds glucose readings within a plus or minus 5-minute tolerance of the target time.

```
SELECT
m.patient_id, m.timestamp AS meal_time,
m.type AS meal_type, m.carbs,
w.label AS window, g.value AS glucose_value

FROM meal m

CROSS JOIN
(VALUES (0,'0 min'),(30,'+30 min'),
(60,'+60 min'),(90,'+90 min'),
(120,'+120 min')) AS w(offset_min, label)

JOIN glucose_level g ON g.patient_id = m.patient_id
AND g.timestamp::timestamp BETWEEN (m.timestamp::timestamp + w.offset_min
* INTERVAL '1 minute' - INTERVAL '5 minutes')
AND (m.timestamp::timestamp + w.offset_min
* INTERVAL '1 minute' + INTERVAL '5 minutes')

ORDER BY m.patient_id, m.timestamp, w.offset_min;
```

	patient_id integer	meal_time timestamp without time zone	meal_type text	carbs real	window text	glucose_value real
1	540	2027-07-04 09:00:00	Breakfast	40	0 min	89
2	540	2027-07-04 09:00:00	Breakfast	40	0 min	90
3	540	2027-07-04 09:00:00	Breakfast	40	+30 min	85
4	540	2027-07-04 09:00:00	Breakfast	40	+30 min	88
5	540	2027-07-04 09:00:00	Breakfast	40	+60 min	130
6	540	2027-07-04 09:00:00	Breakfast	40	+60 min	142
7	540	2027-07-04 09:00:00	Breakfast	40	+90 min	173
8	540	2027-07-04 09:00:00	Breakfast	40	+90 min	172

6.6 Query 3: Bolus Insulin Around Meals

File: 03_bolus_insulin_around_meals.csv

This query links insulin bolus doses to the meals they were intended to cover, using a plus or minus 30-minute window. It reveals whether the patient dosed before or after eating. The bolus_offset_min column shows the timing: a negative value means the patient pre-bolused (administering insulin before eating, which is considered best practice for glucose control), while a positive value means they dosed after the meal (which often leads to higher post-meal glucose spikes). Combined with Query 2, this data allows analysis of whether pre-bolusing leads to better glycemic outcomes.

```
SELECT
m.patient_id, m.timestamp AS meal_time,
m.type AS meal_type, m.carbs,
b.ts_begin AS bolus_time,
b.type AS bolus_type, b.dose,

ROUND(EXTRACT(EPOCH FROM (b.ts_begin::timestamp - m.timestamp::timestamp)) / 60.0, 1)
AS bolus_offset_min


FROM meal m
JOIN bolus_insulin b ON b.patient_id = m.patient_id AND b.ts_begin::timestamp
BETWEEN m.timestamp::timestamp - INTERVAL '30 minutes'
AND m.timestamp::timestamp + INTERVAL '30 minutes'


ORDER BY
m.patient_id,
m.timestamp;
```

	patient_id integer	meal_time timestamp without time zone	meal_type text	carbs real	bolus_time timestamp without time zone	bolus_type text	dose real	bolus_offset_min numeric
1	540	2027-07-04 09:00:00	Breakfast	40	2027-07-04 09:28:22	normal	1.8	28.4
2	540	2027-07-04 19:00:00	Dinner	100	2027-07-04 19:15:08	normal	10	15.1
3	540	2027-07-05 09:00:00	Breakfast	20	2027-07-05 09:25:15	normal	2	25.3
4	540	2027-07-05 11:30:00	Lunch	90	2027-07-05 11:46:24	normal dual	4.5	16.4
5	540	2027-07-05 19:00:00	Snack	1	2027-07-05 18:45:26	normal dual	3.3	-14.6
6	540	2027-07-05 19:00:00	Snack	1	2027-07-05 19:17:01	square dual	3.4	17.0
7	540	2027-07-06 19:30:00	Dinner	50	2027-07-06 19:46:29	normal dual	2.5	16.5
8	540	2027-07-07 05:50:00	Breakfast	40	2027-07-07 05:59:04	normal	4	9.1
9	540	2027-07-08 05:40:00	Breakfast	30	2027-07-08 05:52:51	normal	3	12.9
10	540	2027-07-08 15:00:00	Snack	12	2027-07-08 15:06:55	normal	1.2	6.9
11	540	2027-07-09 10:45:00	Breakfast	90	2027-07-09 11:02:16	normal	8.5	17.3

6.7 Query 4: Finger Stick vs CGM

File: 04_finger_stick_vs_cgm.csv

This query compares manual finger-stick blood glucose readings against the continuous glucose monitor at the same moment, using a plus or minus 5-minute window. This serves as a calibration check: how accurate is the CGM compared to the traditional manual measurement? The difference column shows the discrepancy between the two methods in mg/dL. Finger-stick entries with a value of zero are excluded, as these represent calibration events rather than actual glucose measurements.

```
SELECT
fs.patient_id,
fs.timestamp AS finger_stick_time,
fs.value AS finger_stick_value,
g.timestamp AS cgm_time,
g.value AS cgm_value,
(fs.value - g.value) AS difference

FROM finger_stick fs JOIN glucose_level g ON g.patient_id = fs.patient_id
AND g.timestamp::timestamp
BETWEEN fs.timestamp::timestamp - INTERVAL '5 minutes'
AND fs.timestamp::timestamp + INTERVAL '5 minutes'
WHERE fs.value > 0
ORDER BY fs.patient_id, fs.timestamp;
```

	patient_id integer	finger_stick_time timestamp without time zone	finger_stick_value real	cgm_time timestamp without time zone	cgm_value real	difference real
1	540	2027-07-04 04:30:28	204	2027-07-04 04:31:44	179	25
2	540	2027-07-04 04:30:28	204	2027-07-04 04:26:44	169	35
3	540	2027-07-04 04:41:45	204	2027-07-04 04:46:45	200	4
4	540	2027-07-04 04:41:45	204	2027-07-04 04:41:45	198	6
5	540	2027-07-04 04:41:45	204	2027-07-04 04:36:45	184	20
6	540	2027-07-04 12:27:40	47	2027-07-04 12:31:45	78	-31
7	540	2027-07-04 12:27:40	47	2027-07-04 12:26:45	80	-33
8	540	2027-07-04 12:41:45	47	2027-07-04 12:46:45	56	-9
9	540	2027-07-04 12:41:45	47	2027-07-04 12:41:45	57	-10
10	540	2027-07-04 12:41:45	47	2027-07-04 12:36:45	78	-31
11	540	2027-07-04 18:26:03	155	2027-07-04 18:26:46	126	29
12	540	2027-07-04 18:26:03	155	2027-07-04 18:21:46	123	32
13	540	2027-07-04 18:36:46	155	2027-07-04 18:41:46	149	6
14	540	2027-07-04 18:36:46	155	2027-07-04 18:36:46	147	8
15	540	2027-07-04 18:36:46	155	2027-07-04 18:31:46	130	25
16	540	2027-07-05 00:36:36	499	2027-07-05 00:36:47	387	112
17	540	2027-07-05 00:36:36	499	2027-07-05 00:31:47	381	118
18	540	2027-07-05 01:54:37	491	2027-07-05 01:56:47	397	94

6.8 Query 5: Glucose During Sleep

File: 05_glucose_during_sleep.csv

This query extracts all glucose readings that fall within each sleep period detected by the Basis wearable band. Unlike the other queries that use a plus or minus window around a single timestamp, this one uses a range join: the glucose timestamp must fall between the sleep start (tbegin) and sleep end (tend). This captures every CGM reading across the entire sleep period, which could span six to ten hours. The results reveal overnight glucose patterns, including nocturnal hypoglycemia (dangerous low blood sugar while sleeping) and the dawn phenomenon (a natural rise in glucose in the early morning hours driven by hormonal changes).

```
SELECT
bs.patient_id,
bs.tbegin AS sleep_start,
bs.tend AS sleep_end,
g.timestamp AS glucose_time,
g.value AS glucose_value

FROM basis_sleep bs
JOIN glucose_level g ON g.patient_id = bs.patient_id AND g.timestamp::timestamp
BETWEEN bs.tbegin::timestamp AND bs.tend::timestamp
ORDER BY bs.patient_id, bs.tbegin, g.timestamp;
```

	patient_id integer	sleep_start timestamp without time zone	sleep_end timestamp without time zone	glucose_time timestamp without time zone	glucose_value real
1	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:31:46	107
2	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:36:46	118
3	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:41:46	121
4	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:46:46	127
5	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:51:46	134
6	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 21:56:46	143
7	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:01:46	152
8	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:06:46	162
9	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:11:46	172
10	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:16:46	181
11	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:21:46	190
12	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:26:46	199
13	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:31:46	208
14	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:36:46	216
15	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:41:47	224
16	540	2027-07-04 21:30:00	2027-07-05 10:41:00	2027-07-04 22:46:47	232

6.9 Query 6: Glucose at Bolus Time

File: 06_glucose_at_bolus_time.csv

This query shows the glucose level now the patient injected insulin, using a plus or minus 5-minute window. It answers the question: what glucose reading prompted this bolus? A bolus administered when glucose is 250 mg/dL is a correction dose (bringing dangerously high sugar down), while a bolus at 110 mg/dL is likely a pre-meal dose (preventing a spike from upcoming carbohydrates). This distinction is clinically important for understanding patient behavior and insulin dosing strategies.

```
SELECT
b.patient_id,
b.ts_begin AS bolus_time,
b.type AS bolus_type, b.dose,
g.timestamp AS glucose_time,
g.value AS glucose_at_bolus


FROM bolus_insulin b
JOIN glucose_level g ON g.patient_id = b.patient_id
AND g.timestamp::timestamp BETWEEN b.ts_begin::timestamp - INTERVAL '5 minutes'
AND b.ts_begin::timestamp + INTERVAL '5 minutes'
ORDER BY b.patient_id, b.ts_begin;
```

	patient_id integer	bolus_time timestamp without time zone	bolus_type text	dose real	glucose_time timestamp without time zone	glucose_at_bolus real
1	540	2027-07-04 04:30:28	normal	1.2	2027-07-04 04:31:44	179
2	540	2027-07-04 04:30:28	normal	1.2	2027-07-04 04:26:44	169
3	540	2027-07-04 09:28:22	normal	1.8	2027-07-04 09:31:45	88
4	540	2027-07-04 09:28:22	normal	1.8	2027-07-04 09:26:45	85
5	540	2027-07-04 13:01:48	normal dual	2.7	2027-07-04 13:06:45	85
6	540	2027-07-04 13:01:48	normal dual	2.7	2027-07-04 13:01:45	75
7	540	2027-07-04 13:33:01	square dual	2.8	2027-07-04 13:36:45	116
8	540	2027-07-04 13:33:01	square dual	2.8	2027-07-04 13:31:45	117
9	540	2027-07-04 14:49:49	normal	2.9	2027-07-04 14:51:45	93
10	540	2027-07-04 14:49:49	normal	2.9	2027-07-04 14:46:45	97
11	540	2027-07-04 18:26:03	normal	1.1	2027-07-04 18:26:46	126
12	540	2027-07-04 18:26:03	normal	1.1	2027-07-04 18:21:46	123
13	540	2027-07-04 19:15:08	normal	10	2027-07-04 19:16:46	140
14	540	2027-07-04 19:15:08	normal	10	2027-07-04 19:11:46	141
15	540	2027-07-04 20:11:27	normal	4.5	2027-07-04 20:11:46	130
16	540	2027-07-04 20:11:27	normal	4.5	2027-07-04 20:06:46	134
17	540	2027-07-05 00:36:36	normal	6.8	2027-07-05 00:36:47	387

6.10 Query 7: Exercise Impact on Glucose

File: 07_exercise_impact_on_glucose.csv

This query tracks glucose before exercise and at +30 and +60 minutes after, using the same CROSS JOIN with VALUES pattern as Query 2. Exercise typically lowers blood sugar, sometimes dramatically, because working muscles consume glucose for energy. This query quantifies that effect: by how many mg/dL did glucose drop, and does the magnitude correlate with exercise intensity and duration? The results include both exercise parameters (intensity, duration) and glucose values at each time window, enabling direct comparison.

```
SELECT
e.patient_id,
e.timestamp AS exercise_time,
e.intensity,
e.duration,
w.label AS window,
g.value AS glucose_value

FROM exercise e
CROSS JOIN (
VALUES (0,'at exercise'),
(30,'+30 min'),(60,'+60 min')) AS w(offset_min, label)
JOIN glucose_level g ON g.patient_id = e.patient_id
AND g.timestamp::timestamp BETWEEN (e.timestamp::timestamp + w.offset_min
* INTERVAL '1 minute' - INTERVAL '5 minutes')
AND (e.timestamp::timestamp + w.offset_min
* INTERVAL '1 minute' + INTERVAL '5 minutes')
ORDER BY e.patient_id, e.timestamp, w.offset_min;
```

	patient_id integer	exercise_time timestamp without time zone	intensity integer	duration integer	window text	glucose_value real
1	544	2027-07-02 21:30:00	6	30	at exercise	151
2	544	2027-07-02 21:30:00	6	30	at exercise	152
3	544	2027-07-02 21:30:00	6	30	+30 min	118
4	544	2027-07-02 21:30:00	6	30	+30 min	110
5	544	2027-07-02 21:30:00	6	30	+60 min	90
6	544	2027-07-02 21:30:00	6	30	+60 min	90
7	544	2027-07-03 09:45:00	7	150	at exercise	178
8	544	2027-07-03 09:45:00	7	150	at exercise	190
9	544	2027-07-03 09:45:00	7	150	+30 min	232
10	544	2027-07-03 09:45:00	7	150	+30 min	238
11	544	2027-07-03 09:45:00	7	150	+60 min	235
12	544	2027-07-03 09:45:00	7	150	+60 min	226
13	552	2025-05-27 18:20:00	8	65	at exercise	150

6.11 Query 8: Wearable Sensors at Meals

File: 08_wearable_sensors_at_meals.csv

This query captures skin temperature and galvanic skin response (GSR) from the Basis wearable band at the exact moment the patient ate, using a tight plus or minus 1-minute window. The Basis band records sensor data every minute, so this narrow window is sufficient to find an exact or near-exact match. GSR reflects electrodermal activity, which can indicate stress or autonomic nervous system activation. Skin temperature changes can correlate with metabolic activity. These physiological signals provide additional context around eating events that may help explain variability in post-meal glucose responses.

```
SELECT
m.patient_id, m.timestamp AS meal_time,
m.type AS meal_type, m.carbs,
st.value AS skin_temp,
gsr.value AS gsr_value


FROM meal m
JOIN basis_skin_temperature st ON st.patient_id = m.patient_id
AND st.timestamp::timestamp
BETWEEN m.timestamp::timestamp - INTERVAL '1 minute'
AND m.timestamp::timestamp + INTERVAL '1 minute'
LEFT JOIN basis_gsr gsr ON gsr.patient_id = m.patient_id
AND gsr.timestamp = st.timestamp
ORDER BY m.patient_id, m.timestamp;
```

	patient_id integer	meal_time timestamp without time zone	meal_type text	carbs real	skin_temp real	gsr_value real
1	540	2027-07-04 09:00:00	Breakfast	40	84.13	0.023138
2	540	2027-07-04 09:00:00	Breakfast	40	84.14	0.022796
3	540	2027-07-04 09:00:00	Breakfast	40	84.19	0.021898
4	540	2027-07-04 12:20:00	Lunch	30	82.81	0.00608
5	540	2027-07-04 12:20:00	Lunch	30	83.02	0.008856
6	540	2027-07-04 12:20:00	Lunch	30	83.1	0.009415
7	540	2027-07-04 19:00:00	Dinner	100	87.21	4.565973
8	540	2027-07-04 19:00:00	Dinner	100	86.89	4.435606
9	540	2027-07-04 19:00:00	Dinner	100	86.54	4.31122
10	540	2027-07-05 11:30:00	Lunch	90	83.82	1.556656
11	540	2027-07-05 11:30:00	Lunch	90	83.95	1.936374
12	540	2027-07-05 11:30:00	Lunch	90	83.82	2.60624
13	540	2027-07-05 18:15:00	Dinner	67	90.87	0.018462

7. Conclusion

This project has demonstrated a complete anlysis for the OhioT1DM dataset, transforming six raw XML files into a fully queryable relational database with cross-referencing capabilities. The project processed 204,275 records across 17 tables for six Type 1 Diabetes patients, converting non-standard timestamps to ISO 8601 format, designing a normalized schema with proper foreign key constraints and indexes, and implementing 8 cross-reference queries that connect disparate data streams using timestamp-based windowing.

The timestamp windowing approach, using tolerances of plus or minus 1 to 5 minutes depending on the clinical context, successfully bridges the gap between independently recorded data streams. This technique acknowledges the practical reality that manually recorded clinical measurements rarely share exact timestamps, while still establishing meaningful temporal connections between events.

The nine cross-reference queries cover the most clinically relevant relationships in diabetes management: glucose response to meals, insulin dosing patterns relative to food intake, calibration accuracy between CGM and finger-stick measurements, overnight glucose behavior during sleep, the impact of exercise on blood sugar, and comprehensive meal profiles that combine multiple data streams into a single analytical view.

Course of Study: Informatics / Data Analysis
Completed Hours: 150
Matricola: 558950

Intern: Magamed Imamguliev
Date of Birth: 06/11/2004
Signature:

Academic Supervisor: Armando Ruggeri
Signature:

Firmato digitalmente
da Armando Ruggeri
Data: 21.07.2026
10:17:22 CEST
Organizzazione:
UNIVERSITA' DEGLI
STUDI DI
MESSINA/80004070
837

Academic Tutor: Antonino Galletta
Signature:

Firmato digitalmente
da Antonino Galletta
Data: 09.09.2026
10:18:06 CEST
Organizzazione:
UNIVERSITA' DEGLI
STUDI DI
MESSINA/80004070
837